

Project Brief: Headless OpenFOAM Simulation Engine

1. Objective

Build a robust, fully automated Python wrapper around OpenFOAM that functions as a "Simulation API."

The system must accept a CAD geometry and a configuration file (boundary conditions), autonomously mesh and solve the system, and output structured data (UPDF) for machine learning training.

The Goal: Turn automate data generation to train our foundation models

2. The Problem: The Time Cost of Data Generation

Currently, a major blocker for companies to be able to make use of models—both open source and our own—is pretraining or fine-tuning the models on the *right* data to hit accuracy targets.

Producing the high-fidelity data needed to train our AI is an incredibly time-consuming process. We cannot achieve the volume of data required for a robust model if we are manually setting up cases.

The Solution: We need to build the infrastructure to automate the data generation path. We are not just running simulations; we are building a permanent pipeline that autonomously handles the heavy lifting—geometry intake, meshing, and solving—so we can continuously generate the training assets we need without human intervention.

3. Functional Requirements

→ (prior condition) Geometries will be translated into cad query

Module A: Auto-Mesher

- **Logic:** Programmatic generation of mesh
- **Integration:** Must be modular enough to eventually swap standard meshing for our internal AI-based "Automated meshing" model.

Module B: Solver

- **Execution:** Launch OpenFOAM solvers in a headless environment (no GUI).
- Monitor residuals during the run.
- **Resource Management:** Efficiently distribute tasks across available cores (MPI).

Module C: Data Extraction (The UPDF Link)

- **Post-Processing:** Automatically sample the domain. We do not just want visual snapshots; we need raw tensor data.
- **Conversion:** Convert OpenFOAM internal binary formats into our **Universal Physics Data Format (UPDF)**.
- **Output:** Upload the clean, structured training data to our S3/Cloud storage endpoints.

The main goal of the overall project is to create a ML model that can take **domain design parameters** as input and almost instantly generate the corresponding CFD simulation results. To achieve this, the model is trained on a large dataset of CFD outputs paired with specific domain design parameters, enabling after training to predict CFD results for new, unseen parameter configurations.

Decision: Define number of cases to simulate and the variations for the cases to ensure the dataset covers the necessary design space for the ML model to generalize.

Final result

